Thanks for all the feedback. The site is now fully aligned with Anthropic's updated Exam Guide v0.2 (30 June 2026). **See what changed**. Good luck with studying, Walter DISMISS

RESULTS

Exam Results

788 / 1000

PASSED

22 of 28 correct (79%) · Pass mark: 720

Domain Breakdown

D1	Agentic Architecture & Orchestration	**5/6** (83%) +225pts
D2	Tool Design & MCP Integration	**5/5** (100%) +180pts
D3	Claude Code Configuration & Workflows	**2/6** (33%) +67pts
D4	Prompt Engineering & Structured Output	**5/6** (83%) +167pts
D5	Context Management & Reliability	**5/5** (100%) +150pts

HOW SCORING WORKS

Each domain's percentage correct is multiplied by its weight and scaled to 1000 points. For example, scoring 80% in Agentic Architecture (27% weight) contributes 0.80 × 0.27 × 1000 = 216 points. The pass mark is 720/1000 (72%). If a domain has no questions in the selected scenarios, its weight is redistributed proportionally among assessed domains.

HIDE DETAIL TAKE ANOTHER EXAM

Question-by-Question Review

Q01 · D4 4.2 FEW-SHOT-PROMPTING / FEW-SHOT-STYLE q-4-2-002 · Code Generation with Claude Code

CORRECT

Your Claude Code agent generates unit tests for the TypeScript monorepo. When given detailed instructions alone, it produces tests with inconsistent assertion styles: sometimes using expect().toBe(), sometimes assert.equal(), and occasionally mixing both in the same file. Adding more detailed instructions about assertion style did not fix the problem. What should you do next?

A Switch to a different model that better follows formatting instructions

The issue is not model capability. When detailed instructions alone produce inconsistent formatting, the correct intervention is few-shot examples, not a model change.

D Add 2-4 few-shot examples demonstrating the desired assertion style with reasoning for each testing decision, covering edge cases like async functions and error handling

Few-shot examples are the most effective technique when detailed instructions alone produce inconsistent formatting. Including 2-4 examples with reasoning teaches generalisation to novel patterns, not just pattern-matching. Covering varied scenarios (async, error handling) ensures the model generalises correctly.

C Add 2-4 few-shot examples showing complete test files with the desired assertion style and reasoning for why that style was chosen over alternatives

This is partially correct but incomplete. The examples only cover the assertion style preference. Without covering varied scenarios (async functions, error handling), the model pattern-matches the specific cases shown rather than generalising the style consistently across all test types.

B Add a linter post-processing step to automatically convert all assertions to a single style

Post-processing masks the problem rather than solving it. The model should learn to produce consistent output directly, which few-shot examples achieve more effectively.

WHERE THIS COMES FROM

Lesson 4.4: Few-Shot Prompting (Few-shot examples)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q02 · D3 3.1 CONFIGURATION-SETTINGS / HIERARCHY q-3-1-005 · Developer Productivity Tools INCORRECT

A consultancy works across 12 client projects simultaneously. Each developer has personal preferences (editor keybindings, alias shortcuts) and the consultancy has firm-wide coding standards. Each client project has its own specific conventions. Some client projects have subsystems with additional specialised rules. What is the correct configuration architecture?

A User ~/.claude/CLAUDE.md for personal preferences; project .claude/CLAUDE.md for firm-wide standards and client conventions; directory-level CLAUDE.md or .claude/rules/ with paths for subsystem rules

Personal preferences are per-developer, so they live at user level, which is not shared via version control. Firm-wide standards and client conventions must reach everyone who works on a project, so they belong at the project level, which is committed to the repository and distributed on clone; firm-wide standards can be kept in a shared file and pulled into each project's CLAUDE.md with an @import to stay modular. Putting shared standards at the user level is the classic hierarchy bug, where a colleague who clones a project would not receive them. Subsystem rules belong at the directory level or in path-scoped .claude/rules/. (correct answer)

B User ~/.claude/CLAUDE.md for personal preferences and firm-wide standards; project .claude/CLAUDE.md for client-specific conventions; directory-level CLAUDE.md or .claude/rules/ with paths for subsystem-specific rules

User-level ~/.claude/CLAUDE.md applies only to the individual developer and is not shared via version control. Placing firm-wide standards there means a colleague who clones a client project does not receive them, which is the classic configuration-hierarchy bug. Shared standards belong at the project level. (your answer)

C A single CLAUDE.md at the root of each project containing all four levels of configuration, with clear section headings

Combining all configuration levels into a single file defeats the purpose of the hierarchy. Personal preferences would be version-controlled and shared with the team, firm-wide standards would need duplication across 12 projects, and the file would be bloated with subsystem rules that only apply to specific directories.

D User ~/.claude/CLAUDE.md for everything; use @ path imports to pull in project-specific files from each repository

User-level CLAUDE.md loads for every session across all projects. Importing 12 client project configurations into the user-level file would load all client conventions for every project, wasting tokens and causing conflicting instructions.

WHERE THIS COMES FROM

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (Three-level hierarchy)

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (.claude/rules/)

Q03 · D5 5.4 RATE-LIMITING-QUOTAS / BATCH-VS-REALTIME q-5-4-004 · Customer Support Resolution Agent

CORRECT

A support system processes two request types: password resets (high volume, latency-sensitive, customers wait on screen) and account migrations (low volume, complex, can complete overnight). Both share one synchronous API endpoint at identical priority. At peak hours, migrations consume rate limit capacity and password-reset latency exceeds thresholds. What is the best approach?

C Implement a priority queue where password resets always execute before migration requests, using a single API endpoint

A priority queue on a single endpoint still means both workloads compete for the same rate limit. If the queue is full of migrations, new resets still experience latency waiting for in-flight migration requests to complete.

B Route password resets through the real-time API and account migrations through the batch processing API.

This matches each workload to the appropriate API tier. Password resets need low latency and get real-time processing. Migrations tolerate latency and benefit from batch throughput. The workloads no longer compete for the same rate limits.

D Add more API keys to increase total rate limit capacity so both workloads can run simultaneously without contention

Multiple API keys to increase rate limits may violate terms of service and does not fundamentally solve the architectural mismatch between latency-sensitive and throughput-oriented workloads.

A Pause all account migration requests during business hours and process them only between midnight and 6am

A fixed time window is inflexible and may not align with actual demand patterns. It also delays migrations unnecessarily during off-peak hours when capacity is available.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (Batches API)

Anthropic: Message Batches API ↗

Q04 · D1 1.6 TASK-DECOMPOSITION / ADAPTIVE-DECOMPOSITION q-1-6-002 · Multi-Agent Research System

CORRECT

A consulting firm needs an agent to add a comprehensive test suite to a large legacy codebase. The codebase has no existing tests, unclear dependencies between modules, and the team does not know which areas are most critical. Which task decomposition strategy is most appropriate?

D A multi-pass architecture with per-file analysis and a cross-file integration pass, exactly like a standard code review pipeline.

Multi-pass architecture solves attention dilution for review tasks, but test generation in a legacy codebase needs adaptive exploration: discovering critical modules and dependencies and prioritising coverage. The decomposition must adapt to findings, not follow a fixed per-file pattern.

C Dynamic adaptive decomposition: map the codebase, find the high-impact areas, and adapt the plan as dependencies emerge.

Adding tests to a legacy codebase with unknown dependencies is an open-ended investigation task. Dynamic adaptive decomposition generates subtasks based on what is discovered at each step — first mapping the structure, then identifying critical areas, then adapting the test plan as dependency relationships emerge.

A A fixed sequential pipeline that reviews each file in alphabetical order and generates tests for each one

Fixed sequential pipelines are best for predictable, structured tasks. Adding tests to a legacy codebase with unclear dependencies is an open-ended investigation that requires adaptive exploration to discover which areas are most critical and how dependencies affect test design.

B A single-pass analysis that processes the entire codebase at once and generates a complete test plan

A single-pass analysis of a large codebase would suffer from attention dilution, producing inconsistent coverage. Additionally, a single pass cannot adapt to discoveries about dependencies and critical areas that emerge during exploration.

WHERE THIS COMES FROM

Lesson 1.6: Task Decomposition Strategies (Dynamic adaptive decomposition)

Lesson 1.6: Task Decomposition Strategies (Pattern selection)

Q05 · D2 2.4 MCP-SERVER-INTEGRATION / BUILD-VS-USE q-2-4-001 · Developer Productivity Tools CORRECT

A team needs to integrate with Jira for issue tracking in their Claude Code workflow. A developer proposes building a custom MCP server. What is the correct first step?

D Add the Jira integration to ~/.claude.json so each developer can configure it independently.

A team-wide integration should be in project-level .mcp.json so it is version-controlled and shared. User-level ~/.claude.json is for personal/experimental servers.

A Build a custom MCP server with the exact API endpoints the team needs.

Building custom is premature. Community MCP servers for Jira already exist and cover standard use cases. Custom builds should be reserved for team-specific workflows.

B Evaluate existing community MCP servers for Jira and only build custom if they cannot handle team-specific workflows.

Community servers should always be the first choice for standard integrations. They are maintained, tested, and cover common use cases. Custom builds are justified only when community servers cannot handle team-specific requirements.

C Use the Jira REST API directly from Bash commands instead of MCP.

Direct API calls bypass the MCP tool interface, losing the benefits of tool descriptions, structured responses, and agent-native integration.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Build-vs-use)

Claude Code: MCP Server Configuration ↗

Q06 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / SCOPED-CROSS-ROLE q-2-3-001 · Developer Productivity Tools

CORRECT

A synthesis agent frequently returns control to the coordinator for simple fact verification, adding 2-3 round trips per task and 40% latency. Analysis shows 85% of verifications are simple lookups. What is the most effective solution?

A Increase the coordinator's parallelism so it can process verification requests faster.

Faster processing does not eliminate unnecessary round trips. The latency comes from the routing overhead itself, not the coordinator's speed.

C Add a coordinator-level cache of verification results so repeated lookups return instantly, removing most of the round-trip latency.

Caching helps with repeated lookups but does not address the fundamental round-trip overhead for first-time verifications, which are the majority.

B Give the synthesis agent a scoped verify_fact tool for simple lookups, escalating only complex checks to the coordinator.

A scoped cross-role tool handles the 85% simple case directly, eliminating round-trip latency. Complex cases still route through the coordinator for proper handling.

D Remove fact verification from the synthesis workflow to eliminate the latency entirely.

Removing verification compromises output quality. The goal is to make verification faster, not to skip it.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (Scoped cross-role tools)

Q07 · D4 4.1 SYSTEM-PROMPTS / SEVERITY-EXAMPLES q-4-1-002 · Code Generation with Claude Code

CORRECT

Your Claude Code review prompt for the TypeScript monorepo classifies severity using prose descriptions like 'critical means the code is dangerous' and 'minor means the code is slightly suboptimal'. Developers complain that identical code patterns receive different severity ratings

across runs. What is the most effective improvement?

A Replace prose severity descriptions with concrete TypeScript code examples for each severity level.

Severity calibration with concrete code examples produces consistent classification across invocations. Prose descriptions are inherently ambiguous and leave the model to interpret severity differently each time.

C Lower the model temperature to 0 to ensure deterministic severity ratings

While lower temperature reduces randomness, it does not address the fundamental problem: the model has no concrete reference for what each severity level looks like. Code examples provide that reference.

D Add a second model pass that re-evaluates each finding's severity to catch inconsistencies

A second pass using the same vague prose criteria will produce the same inconsistency. The criteria themselves must be improved with concrete code examples before adding verification layers.

B Add a confidence threshold so only findings above 90% confidence are reported, filtering out uncertain severity ratings

Self-reported confidence scores are poorly calibrated. Confidence-based filtering does not fix the root cause, which is ambiguous severity criteria. Explicit code examples are the correct fix.

WHERE THIS COMES FROM

Lesson 4.1: System Prompts with Explicit Criteria (Severity calibration)

Q08 · D1 1.7 SESSION-STATE-RESUMPTION / FRESH-START q-1-7-004 · Code Generation with Claude Code

CORRECT

A Claude Code agent has been working on a feature branch for 45 minutes and has accumulated extensive context about the codebase. The developer notices that several tool results from early in the session (file reads from 40 minutes ago) are now stale because a colleague pushed changes to those files. The agent is making recommendations based on the outdated file contents. What is the best recovery strategy?

B Start a completely new session with no context from the previous session

Starting fresh discards 45 minutes of accumulated context and architectural understanding. The agent would need to rediscover all the insights it already found about the codebase.

A Continue in the current session and simply ask the agent to re-read the files a colleague changed, so it picks up the latest contents.

Re-reading updates the file contents, but the stale results remain in the context window. The agent may still reference the outdated content from earlier in the conversation, leading to contradictory reasoning.

D Use fork_session to create a new branch that excludes the stale tool results

fork_session creates a branch from the current state, which includes the stale tool results. It does not allow selective exclusion of context. It is designed for divergent exploration, not for stale context recovery.

C Start a fresh session with a summary of the key findings and decisions, then read the changed files for current state.

Fresh start with summary injection is the prescribed recovery strategy for stale tool results. It preserves the valuable insights and decisions from the previous session while eliminating the stale data. Reading the changed files in the new session ensures the agent works from current state.

WHERE THIS COMES FROM

Lesson 1.7: Session State and Resumption (Stale context)

Lesson 1.7: Session State and Resumption (Decision matrix)

Q09 · D3 3.1 CONFIGURATION-SETTINGS / PATH-RULES q-3-1-002 · Code Generation with Claude Code

INCORRECT

A large TypeScript monorepo has a root CLAUDE.md with universal coding standards and a .claude/rules/ directory with topic-specific rule files. A developer notices that conventions from testing.md in .claude/rules/ are loading even when editing API handler files, consuming unnecessary tokens. The testing.md file has no YAML frontmatter. What should the developer do to fix this?

D Use an @./.claude/rules/testing.md line in the root CLAUDE.md so the file only loads when the import is reached

@ path imports load eagerly when the CLAUDE.md is loaded — the imported file's content is inlined at that moment, regardless of which file the developer is editing. They are a source-organisation mechanism, not a conditional-loading mechanism. Path-scoped frontmatter in .claude/rules/ is the only way to make rule files load only for matching paths.

C Add YAML frontmatter with a paths field of test-file globs to testing.md so it loads only for test files

Without YAML frontmatter, rule files in .claude/rules/ load for all sessions. Adding a paths field with glob patterns restricts loading to sessions where the developer is editing matching files. This is the correct approach for token-efficient, conditional convention loading. (correct answer)

B Add the testing conventions to the root CLAUDE.md instead, since rules files cannot be path-scoped

Rules files absolutely support path scoping via YAML frontmatter with a paths field. Moving conventions to root CLAUDE.md would make the token efficiency problem worse, not better, because root CLAUDE.md loads for every session.

A Move testing.md out of .claude/rules/ and into a directory-level CLAUDE.md inside the test folder

Test files are co-located with source files throughout 200+ packages in this monorepo. A directory-level CLAUDE.md in a single test folder would not cover test files in other directories, and creating one in every directory would be unmaintainable. (your answer)

WHERE THIS COMES FROM

Lesson 3.3: Path-Specific Rules (Path-specific rules)

Lesson 3.1: CLAUDE.md Hierarchy and Scoping (.claude/rules/ directory)

Q10 · D1 1.5 AGENT-SDK-HOOKS / PRETOOLUSE-ENFORCEMENT q-1-5-001 · Customer Support Resolution Agent

CORRECT

An agent occasionally processes international transfers without required compliance checks. The compliance team requires 100% enforcement of anti-money laundering (AML) checks before any international transfer. What is the correct approach?

B Implement a PreToolUse hook that blocks transfer execution until AML verification returns a pass result

A hook intercepts the outgoing tool call before execution and physically blocks it until the compliance check passes. This provides the deterministic guarantee that AML rules require.

A Add detailed AML check instructions to the system prompt with examples of correct behaviour

Prompt instructions provide probabilistic compliance. With international transfers and AML regulations, a single failure could result in legal penalties. 100% enforcement requires deterministic guarantees.

C Add a PostToolUse hook to flag completed transfers that skipped AML checks

PostToolUse hooks run after execution. By the time the hook fires, the non-compliant transfer has already been processed. Prevention is required, not detection.

D Train the agent with few-shot examples showing the correct AML verification workflow

Few-shot examples improve accuracy but do not guarantee 100% compliance. Regulatory requirements demand deterministic enforcement via hooks.

WHERE THIS COMES FROM

Lesson 1.5: Agent SDK Hooks (PreToolUse hooks)

Anthropic: Agent SDK Hooks ↗

Q11 · D3 3.5 SLASH-COMMANDS-MCP / EXAMPLES q-3-5-001 · Code Generation with Claude Code CORRECT

A developer describes a code transformation in prose. Claude Code interprets it differently each time, producing inconsistent results. What technique should the developer try first?

D Write a test suite first and iterate by sharing test failures

Test-driven iteration is effective but is a more heavyweight approach. Concrete examples are the faster first step when the issue is inconsistent interpretation of a known transformation.

A Rewrite the prose description with more precise language and technical terminology

More precise prose still relies on the model's interpretation of natural language. If interpretation is inconsistent, more prose will not solve the root cause.

C Use the interview pattern to have Claude ask clarifying questions before implementing

The interview pattern is best for unfamiliar domains where the developer might miss considerations. When the developer knows the exact transformation but the model interprets it inconsistently, examples are the direct fix.

B Provide 2-3 concrete input/output examples showing the exact before and after transformation

Concrete examples eliminate interpretation ambiguity. The model generalises from examples more reliably than from descriptions. This is the documented first-line technique for inconsistent interpretation.

WHERE THIS COMES FROM

Lesson 3.5: Iterative Refinement Techniques (Example-based communication)

Lesson 3.5: Iterative Refinement Techniques (Technique hierarchy)

Q12 · D3 3.4 PERMISSIONS-SECURITY / ALLOWED-TOOLS q-3-4-004 · Code Generation with Claude Code

INCORRECT

A junior developer is using Claude Code to refactor a payment processing module. The team lead wants to ensure Claude Code cannot accidentally delete production configuration files or modify the database migration directory during the refactoring session. What is the most appropriate approach?

A Instruct the developer to use plan mode so Claude Code will ask for approval before each file change

Plan mode helps with evaluating strategies for complex tasks, but it does not provide file-level access control. Even in plan mode, the model can still access all tools and directories once execution begins.

C Add a rule in CLAUDE.md stating 'Never modify files in the config/ or migrations/ directories'

CLAUDE.md instructions rely on the model's compliance and are not deterministic safeguards. For production-critical protection, tool-level permission restrictions provide reliable enforcement rather than natural language instructions. (your answer)

D Set the repository to read-only mode at the filesystem level before starting the session

Making the entire repository read-only would prevent Claude Code from making any changes, including the legitimate refactoring of the payment module. The requirement is selective protection, not blanket read-only access.

B Use allowedTools to limit the session to Read, Grep, Glob, and Write within the payment module

allowedTools provides tool-level permission control that can restrict which tools are available and their scope. By limiting write access to the payment module directory and keeping read access for context, the team lead ensures Claude Code cannot modify production config or migration files. (correct answer)

WHERE THIS COMES FROM

Claude Code: Settings ↗

Q13 · D1 1.2 ORCHESTRATION-PATTERNS / COORDINATOR q-1-2-002 · Multi-Agent Research System CORRECT

A consulting firm's multi-agent research system has a coordinator that always invokes the full pipeline of five subagents (web search, document analysis, data extraction, synthesis, and formatting) for every query, including simple factual lookups that only need web search. This adds unnecessary latency and cost. What is the correct architectural fix?

C Add a caching layer so subagents that have no work to do return immediately

Caching does not address the architectural issue. Invoking subagents that are not needed adds API call overhead and latency regardless of whether results are cached. The coordinator should not invoke unnecessary subagents in the first place.

B Allow subagents to communicate directly with each other so they can skip unnecessary steps in the pipeline

Direct subagent communication violates the hub-and-spoke architecture. All communication must flow through the coordinator. The fix is smarter coordinator routing, not bypassing the coordinator.

D Split into two separate systems: one for simple queries and one for complex queries

Maintaining two separate systems creates unnecessary complexity. The coordinator's role is precisely to analyse query requirements and make routing decisions. A single coordinator with dynamic selection handles both cases.

A Have the coordinator select which subagents to invoke per query, not always the full pipeline.

The coordinator should analyse query requirements and decide which subagents are needed. A simple factual lookup only needs the web search agent, not the full five-agent pipeline. Dynamic selection reduces latency and cost for straightforward requests.

WHERE THIS COMES FROM

Lesson 1.2: Multi-Agent Orchestration (Coordinator responsibilities)

Q14 · D5 5.6 PRODUCTION-RELIABILITY / CONFLICT-HANDLING q-5-6-004 · Multi-Agent Research System

CORRECT

A multi-agent research system finds conflicting data on a pharmaceutical compound's efficacy. A 2022 peer-reviewed clinical trial reports 78% efficacy, while a 2024 preprint reports 45% efficacy with a larger sample size. The synthesis agent must produce a recommendation for a medical advisory board. Which approach correctly handles the conflicting sources?

C Present both findings with full provenance: source identity, publication date, peer-review status, sample size, methodology, and any methodological differences that may explain the discrepancy. Let the advisory board weigh the evidence

For a medical advisory board, preserving full provenance is critical. The board needs to weigh peer-review status, sample size, methodology differences, and temporal context. Annotating both sources with complete metadata enables informed human judgment on conflicting scientific evidence.

B Average the two values (61.5%) and present it as the best estimate with a note about variance between studies

Averaging conflicting scientific results is methodologically invalid. The studies may have different populations, methodologies, or endpoints. The average has no scientific meaning and obscures the actual disagreement.

A Use the 2024 preprint as it has a larger sample size and is more recent, noting it supersedes the 2022 study

Recency and sample size alone do not determine reliability. The preprint has not undergone peer review, and simply picking one source discards valuable context that the advisory board needs for informed decision-making.

D Exclude both findings and report that no reliable consensus exists, recommending further research before any conclusions

Excluding available evidence is unhelpful. The advisory board can make informed decisions with properly attributed conflicting data. Withholding findings because they conflict reduces the board's ability to assess the situation.

WHERE THIS COMES FROM

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflict handling)

Lesson 5.6: Information Provenance and Multi-Source Synthesis (Conflicts intact)

Q15 · D5 5.3 LONG-CONVERSATIONS / ACCESS-VS-EMPTY q-5-3-002 · Multi-Agent Research System CORRECT

A research subagent queries an academic journal database for articles on renewable energy policy. The database responds successfully but returns zero matching articles. Meanwhile, a second subagent querying a government statistics API receives a connection timeout. The coordinator currently handles both cases identically by retrying three times. What should change?

B Add exponential backoff to all retries so the subagents wait progressively longer between attempts

Backoff improves retry behaviour for transient failures but does not address the fundamental issue: the journal query does not need retries at all. It returned a valid response indicating no matches exist.

A Increase the retry count to five for both cases to give transient failures more time to resolve

More retries do not fix the core problem. The journal database returned a valid empty result — it successfully searched and found nothing. Retrying it wastes time and API calls on an operation that already completed correctly.

D Distinguish access failures from valid empty results: retry the API timeout as a transient failure, but accept the zero-match journal response as a valid finding and annotate the coverage gap

Access failures (timeouts, connection errors) indicate the tool could not reach the data source and warrant retry. Valid empty results mean the tool reached the source and found no matches — this IS the answer. Conflating these leads to unnecessary retries on valid results and missed coverage annotations.

C Remove retries entirely and immediately escalate both failures to a human operator for investigation

This is disproportionate. The API timeout is a transient failure that retries can resolve. The empty journal result is not a failure at all — it is a valid answer. Neither case requires human escalation.

WHERE THIS COMES FROM

Lesson 5.3: Error Propagation in Multi-Agent Systems (Access failure vs empty)

Lesson 5.3: Error Propagation in Multi-Agent Systems (Local recovery for transient)

Q16 · D1 1.3 SUBAGENT-INVOCATION-CONTEXT / SCOPED-TOOLS q-1-3-013 ·

Customer Support Resolution Agent CORRECT

An architect is designing a customer support system using the Claude Agent SDK. The system needs a coordinator agent that can delegate to a billing specialist and a technical support specialist. Each specialist needs different tools. How should the architect configure the AgentDefinitions?

D Give the coordinator all tools and have it handle all requests directly, eliminating the need for subagents

A single agent with all tools has a higher error rate due to tool selection confusion. Specialist subagents with scoped tools produce more reliable results. As a rough guide, aim for a small set of around 4–5 tools per

agent.

C Create the specialists as separate API endpoints and have the coordinator call them via HTTP rather than the Task tool

Using HTTP calls instead of the Task tool bypasses the Agent SDK's built-in orchestration capabilities. The Task tool is the designed mechanism for subagent invocation and provides proper context management.

A Create one AgentDefinition with all tools from both specialists, and use prompt instructions to tell the agent which tools to use in each context

Combining all tools into one agent violates the principle of scoped tool access. With too many tools, the agent is more likely to use the wrong tool. Each specialist should have only the 4–5 tools relevant to its role.

B Create separate AgentDefinitions for each specialist with scoped tools (4–5 tools each), and give the coordinator the Task tool to spawn them

Each specialist gets its own AgentDefinition with its tools restricted to the 4–5 tools relevant to its role. The coordinator has the Task tool (renamed to Agent in current Claude Code v2.1.63; 'Task' still works as a backward-compatible alias) in its allowedTools so it can spawn specialists. This enforces separation of concerns and scoped tool access.

WHERE THIS COMES FROM

Lesson 1.4: Workflow Enforcement and Handoff (Enforcement spectrum: scoped tools)

Lesson 1.3: Subagent Invocation and Context Passing (Task tool)

Anthropic: Claude Agent SDK Overview ↗

Q17 · D4 4.5 BATCH-PROCESSING / PROMPT-CACHING q-4-5-006 · Code Generation with Claude Code

CORRECT

Your Claude Code setup uses a complex system prompt with detailed review instructions, code examples, and few-shot demonstrations that total 4,000 tokens. Each code review request adds 500-2,000 tokens of code to review. The team wants to optimise costs since the same review instructions are used for every request. What is the most effective optimisation?

D Reduce the number of few-shot examples from 4 to 1 and lower the temperature to compensate for the reduced guidance

Lower temperature does not compensate for reduced examples. Fewer examples degrade quality. Prompt caching preserves the full prompt while reducing costs.

C Switch all code reviews to the Message Batches API for 50% cost savings

Code reviews in Claude Code are interactive, blocking workflows. The Batches API's 24-hour window makes it unsuitable for developer-facing reviews.

B Enable prompt caching with the static review instructions and examples first and the code-to-review at the end.

Prompt caching caches the static prefix and reuses it across requests. With 4,000 tokens of static content reused for every review, caching provides significant cost savings without sacrificing review quality. Dynamic code goes at the end so the prefix remains cacheable.

A Shorten the system prompt by removing few-shot examples to reduce per-request token costs

Removing few-shot examples degrades review quality. The 4,000-token static prompt is the ideal candidate for caching, not trimming.

WHERE THIS COMES FROM

Lesson 4.5: Batch Processing and Prompt Optimisation (Prompt caching layout)

Anthropic: Prompt Caching ↗

Q18 · D1 1.4 WORKFLOW-ENFORCEMENT-HANDOFF / GRACEFUL-DEGRADATION q-1-4-009 ·

Customer Support Resolution Agent INCORRECT

A customer support agent's loop terminates when cumulative token usage exceeds a budget. During a complex billing investigation the budget is exhausted mid-task, after data gathering but before resolution, with stop_reason still `tool_use`. What architectural change addresses this?

D Switch to a cheaper model so each iteration uses fewer tokens, letting the agent fit more iterations into the same budget and finish the task.

A cheaper model may reduce the quality of the investigation itself. The architectural fix is graceful degradation when constraints are reached, not lowering the quality of work to fit within arbitrary limits.

A Double the token budget to ensure the agent always has enough tokens to complete complex investigations

Doubling the budget is an arbitrary fix that may still be insufficient for edge cases and wastes tokens on simple requests. The fundamental issue is that the budget terminates the loop even when stop_reason indicates the agent has more work to do. (your answer)

B When the budget is nearly gone and stop_reason is still 'tool_use', summarise progress and escalate to a human.

When constraints prevent completion, the agent should degrade gracefully rather than terminating abruptly. Summarising progress and performing a structured handoff (customer ID, summary, root cause so far, recommended next steps) ensures no work is lost and the human agent can continue from where the AI left off. (correct answer)

C Remove the token budget entirely and rely solely on the iteration cap for safety

Removing the token budget eliminates a valuable cost control mechanism. The issue is not having a budget, but how the system handles budget exhaustion during active work.

WHERE THIS COMES FROM

Lesson 1.1: Agentic Loops (Agentic loop lifecycle)

Lesson 1.4: Workflow Enforcement and Handoff (Structured handoff protocols)

Q19 · D2 2.1 TOOL-SCHEMA-DESIGN / SYSTEM-PROMPT q-2-1-002 · Developer Productivity Tools CORRECT

An agent has two tools: `analyze_content` ('Analyses content') and `extract_web_results` ('Extracts data from web pages'). After a system prompt update added 'Always analyse content before responding', the agent started routing web-extraction tasks to `analyze_content` despite the unchanged descriptions. What is the most likely cause and fix?

C The analyze_content tool should be renamed to something less generic, such as summarize_document, to avoid all future conflicts.

Renaming is a valid longer-term improvement but does not address the immediate cause: the system prompt instruction creating a keyword match. The prompt conflict would persist with any tool whose name overlaps with prompt wording.

A The system prompt's 'analyse content' phrasing keyword-matches the analyze_content tool; rephrase it to remove the overlap.

Keyword-sensitive instructions in system prompts can create unintended tool associations that override well-written descriptions. The phrase 'analyse content' directly matches the tool name, so the model favours it regardless of the task. The fix is to review and rephrase the system prompt after any update.

D The tool descriptions have degraded over time and need to be rewritten with clearer boundaries between the two tools.

The question states tool descriptions were not changed and routing was correct before the system prompt update. The descriptions are not the root cause — the new system prompt instruction is.

B The tool descriptions need few-shot examples added to the system prompt showing when to use each tool.

Few-shot examples add token overhead and do not address the root cause. The system prompt instruction is directly triggering the wrong tool via keyword association, and adding more content to the system prompt may compound the issue.

WHERE THIS COMES FROM

Lesson 2.1: Tool Interface Design (System prompt interactions)

Q20 · D4 4.3 STRUCTURED-OUTPUT / ENUM-FIELDS q-4-3-004 CORRECT

The moderation system's classification schema has a 'category' field defined as a free-text string. Auditors find 47 different category values in production data, including 'hate speech', 'Hate Speech', 'hate-speech', 'hateful content', and 'hate_speech' — all intended to be the same category. This makes downstream analytics and routing unreliable. What is the best schema fix?

A Add a post-processing normalisation step that maps all variations to canonical category names

Post-processing normalisation adds complexity and requires constant maintenance as new variations appear. The schema should prevent the problem at the source rather than patching it downstream.

C Add detailed instructions to the prompt listing the exact category names and their capitalisation so the model always emits the canonical string.

Prompt instructions are probabilistic. The model may still produce variations despite instructions. Schema-level enforcement via enums is deterministic and cannot be overridden.

D Add few-shot examples showing the correct category formatting for each type

Few-shot examples improve consistency but cannot guarantee exact string matching. Schema enums are the correct mechanism for constraining categorical output to a fixed set of values.

B Change 'category' from free-text to an enum with values like 'hate_speech', 'spam', and 'harassment', plus an 'other' option.

Enum fields constrain the model to predefined values, eliminating spelling and formatting variations. Including 'other' as an enum value handles edge cases without allowing free-text drift. With strict: true, the schema guarantees only valid enum values are returned.

WHERE THIS COMES FROM

Lesson 4.2: Structured Output with Tool Use (Enum schema)

Q21 · D4 4.2 FEW-SHOT-PROMPTING / CONSTRUCTION q-4-2-006 · Code Generation with Claude Code

CORRECT

Your Claude Code agent generates API endpoint implementations. You provide detailed instructions specifying error handling conventions, but the generated code inconsistently handles async errors: sometimes using try/catch, sometimes using .catch(), and sometimes omitting error handling entirely. Adding more detailed instructions did not resolve the inconsistency. What is the most effective next step?

B Add 2-4 few-shot examples demonstrating the correct error handling pattern across varied async scenarios, with reasoning for each choice.

When detailed instructions alone fail to produce consistent formatting, few-shot examples with reasoning are the correct escalation. Covering varied async scenarios prevents the model from pattern-matching only the specific cases shown and teaches it to generalise the error handling style.

A Add a linting rule that flags and rejects generated code whose error handling deviates from the target style, catching nonconforming output after generation.

Rejecting nonconforming output gates results after generation but never teaches the model to produce the consistent style itself, so novel cases keep failing. Few-shot examples with reasoning fix the inconsistency at source.

C Set temperature to 0 to eliminate the randomness causing inconsistent error handling styles

Temperature does not fix ambiguous criteria. If the instructions do not clearly demonstrate the preferred pattern, low temperature just makes the model consistently pick the same ambiguous interpretation, not necessarily the correct one.

D Add a post-generation review step that rewrites any incorrect error handling patterns

Post-processing adds cost and complexity. It is more effective to teach the model the correct pattern upfront with few-shot examples than to fix incorrect output afterwards.

WHERE THIS COMES FROM

Lesson 4.4: Few-Shot Prompting (Effective examples)

Anthropic: Multishot (Few-Shot) Prompting ↗

Q22 · D5 5.2 PROMPT-CACHING / AMBIGUOUS-MATCHING q-5-2-004 · Customer Support Resolution Agent

CORRECT

A customer support agent searches for a customer named 'Sarah Johnson' and the lookup tool returns three matching accounts with the same name but different addresses and account ages. The agent selects the most recently active account and proceeds with the refund. Later, the customer calls back because the refund was applied to the wrong account. What should the agent have done differently?

A Selected the account with the oldest creation date, since long-standing customers are the most likely callers to a support line.

Account age is an unreliable heuristic for identifying the correct customer. Any selection based on heuristics risks choosing the wrong account.

D Escalated to a human agent immediately because the system returned ambiguous results.

Ambiguous search results are a routine scenario that the agent can resolve by asking for clarification. Escalation is not warranted when the agent can gather additional information to disambiguate.

B Asked the customer for additional identifying information to disambiguate the matching accounts.

When multiple customers match a search query, the agent must ask for additional identifiers rather than selecting based on heuristics. This ensures the correct account is identified before any action is taken.

C Processed the refund across all three accounts to ensure the correct customer received it.

Processing refunds on incorrect accounts creates financial discrepancies and additional work to reverse the erroneous transactions. The agent must identify the correct account first.

WHERE THIS COMES FROM

Lesson 5.2: Escalation and Ambiguity Resolution (Ambiguous customer matching)

Q23 · D3 3.2 CLAUDE-MD-FILES / SKILLS-FRONTMATTER q-3-2-002 · Developer Productivity Tools INCORRECT

A platform engineering team creates a /security-audit skill that scans the codebase for vulnerabilities. The skill should be available to every developer on the project, must not be able to modify any files, and produces extensive analysis output. Which configuration is correct?

B Place the skill in .claude/commands/ with allowed-tools restricting it to Read, Grep, and Glob

While .claude/commands/ is project-scoped (and equivalent to .claude/skills/ — both paths create identical /commands), this configuration is missing context: fork for isolating the extensive analysis output. Frontmatter features like context: fork and allowed-tools require a SKILL.md file in .claude/skills/, making that the better location for this use case.

D Place the skill in .claude/skills/ with a SKILL.md containing allowed-tools: ["Read", "Grep", "Glob"] and context: fork in the frontmatter

.claude/skills/ is project-scoped and shared via version control, so every developer gets it. allowed-tools restricts the skill to read-only tools, preventing file modifications. context: fork isolates the extensive analysis output from the main conversation. This satisfies all three requirements. (correct answer)

C Place the skill in CLAUDE.md as an always-loaded security scanning procedure

CLAUDE.md is for universal, always-loaded standards, not on-demand task-specific workflows. A security audit is invoked when needed, not applied to every session. Placing it in CLAUDE.md wastes tokens in sessions where no audit is needed.

A Place the skill in ~/.claude/skills/ with allowed-tools restricting it to Read, Grep, and Glob, and context: fork in the frontmatter

~/.claude/skills/ is user-scoped and not shared via version control. The requirement states every developer on the project needs access, so it must be project-scoped. (your answer)

WHERE THIS COMES FROM

Lesson 3.2: Custom Slash Commands and Skills (Skills frontmatter)

Lesson 3.2: Custom Slash Commands and Skills (Project vs user scoping)

Claude Code: Skills ↗

Q24 · D4 4.6 MULTI-PASS-REVIEW / SELF-REVIEW-BIAS q-4-6-002 · Code Generation with Claude Code

INCORRECT

Your Claude Code setup reviews generated code by asking the same model instance to critique its own output immediately after generation, within the same conversation. The team notices the reviews are overly favourable and miss bugs that an external reviewer would catch. What is the root cause and correct fix?

B Enable extended thinking so the model reasons more carefully about potential issues in its own code

Extended thinking does not solve the self-review limitation. The model still retains its reasoning context and is biased towards its own output. Extended thinking makes the existing reasoning deeper, not more independent.

D Lower the temperature during the review phase to make the model more precise and less likely to approve questionable code

Temperature affects response variability, not the fundamental bias of self-review. The model will still be biased towards its own output regardless of temperature settings. An independent instance is the correct architectural fix.

A The model needs more detailed review instructions; add a comprehensive checklist of what to look for during self-review

More detailed instructions do not overcome the fundamental limitation. The model retains its reasoning context from generation and is less likely to question its own decisions regardless of how detailed the checklist is. (your answer)

C Split into per-file local analysis passes for consistent depth, then a separate cross-file integration pass.

A model reviewing its own output in the same session retains reasoning context and is less likely to question its own decisions. An independent instance without prior context catches more subtle issues because it approaches the code fresh. (correct answer)

WHERE THIS COMES FROM

Lesson 4.6: Multi-Instance Review and Output Validation (Self-review limitation)

Q25 · D2 2.3 TOOL-DISTRIBUTION-CHOICE / TOOL-CHOICE q-2-3-003 · Developer Productivity Tools CORRECT

A developer productivity platform uses an automated documentation agent. The agent must always extract metadata from source files before generating documentation, but in testing it sometimes skips the metadata extraction step and produces documentation from incomplete information. What is the correct tool_choice configuration to enforce the mandatory first step?

A Set tool_choice to 'auto' and add a system prompt instruction: 'You must call extract_metadata before any other tool.'

With 'auto', the model decides whether to call a tool at all. A system prompt instruction is advisory, not enforced — the model can still skip the step. This does not guarantee the mandatory first step.

D Remove all other tools except extract_metadata from the agent's configuration so it has no choice.

This prevents the agent from performing any other task after metadata extraction. The agent needs access to documentation generation tools for subsequent steps. Forced tool selection achieves the constraint

without crippling the agent.

C Set tool_choice to forced selection with the specific tool name extract_metadata for the first turn, then switch to 'auto' for subsequent turns.

Forced selection {'type': 'tool', 'name': 'extract_metadata'} ensures the model must call this specific tool. The model cannot skip it or choose a different tool. After the first turn completes, switching to 'auto' allows normal operation for documentation generation.

B Set tool_choice to 'any' so the model must call a tool, relying on the tool description to guide it to extract_metadata first.

'any' forces the model to call a tool but lets it choose which one. It may select a different tool first, still skipping extract_metadata. This guarantees a tool call but not the correct tool call.

WHERE THIS COMES FROM

Lesson 2.3: Tool Distribution and Tool Choice (tool_choice configuration)

Anthropic: Tool Use Documentation ↗

Q26 · D2 2.4 MCP-SERVER-INTEGRATION / ENV-VARS q-2-4-005 · Developer Productivity Tools CORRECT

A platform engineering team wants to share an MCP server for their internal ticketing system across all developers using Claude Code. The server requires an API token unique to each developer. Where should the MCP server be configured, and how should credentials be managed?

D Create a shared .env file in the repository with all developer tokens and reference it from .mcp.json.

A shared .env file with all developer tokens still commits credentials to version control and creates a single file containing every team member's secrets, compounding the security risk.

A Add the server to .mcp.json in the project root with each developer's API token hard-coded in the configuration.

Hard-coding individual tokens in .mcp.json commits credentials to version control, which is a security risk. Each developer has a different token, so a single hard-coded value would not work for the team.

C Have each developer add the server to their personal ~/.claude.json with their API token.

User-level ~/.claude.json is for personal or experimental servers. A team-wide integration should be in project-level .mcp.json so it is version-controlled and consistently available to all developers.

B Add the server to .mcp.json using ${TICKETING_API_TOKEN} expansion so each developer sets their own token.

Project-level .mcp.json is version-controlled and shared, making the server available to everyone. Environment variable expansion keeps individual credentials out of version control while letting each developer authenticate with their own token.

WHERE THIS COMES FROM

Lesson 2.4: MCP Server Integration (Environment variable expansion)

Lesson 2.4: MCP Server Integration (Scoping hierarchy)

Claude Code: MCP Server Configuration ↗

Q27 · D5 5.3 LONG-CONVERSATIONS / CONTEXT-DEGRADATION q-5-3-004 · Multi-Agent Research System

CORRECT

A multi-agent system runs a 45-minute deep research workflow. Around the 30-minute mark, the coordinator's synthesis quality drops: it refers to 'the key findings' instead of specific statistics it cited earlier, and misattributes a claim from the financial subagent to the news subagent. No token limits or errors have been hit. What is the most likely diagnosis and correct mitigation?

B Context degradation: early results are buried deep in a long context. Consolidate key findings into a structured block near the end.

Context degradation occurs when important information is buried deep in a long conversation. The model's attention to early content diminishes as new content accumulates. Consolidating key findings into a recent, structured block keeps them in the model's effective attention window without losing the specifics.

C The token limit has been silently exceeded and the API is truncating early messages. Upgrade to a larger context window model

The question states no token limits have been hit. Context degradation occurs well before token limits are reached — it is an attention quality issue, not a capacity issue.

D The subagents are returning inconsistent data, confusing the coordinator. Add data validation to each subagent's output before it reaches the coordinator

The coordinator previously cited these same findings correctly earlier in the session. The issue is not data quality from subagents but the coordinator's degrading attention to that data over time.

A The model is experiencing 'fatigue' from a long session and needs a cooldown period before continuing

LLMs do not experience fatigue. Each inference is independent. The degradation is caused by context window dynamics, not model tiredness.

WHERE THIS COMES FROM

Lesson 5.4: Codebase Exploration and Context Degradation (Context degradation)

Lesson 5.4: Codebase Exploration and Context Degradation (Summary injection)

Q28 · D3 3.4 PERMISSIONS-SECURITY / PLAN-MODE q-3-4-002 · Code Generation with Claude Code CORRECT

A developer is migrating a monorepo from CommonJS to ES modules: 80+ files, multiple valid strategies, knock-on effects in build tooling, test configuration, and `package.json`. They start in direct execution mode and begin converting files. Halfway through, the chosen import resolution strategy breaks the test runner. What went wrong?

D The developer should have written a complete test suite first and iterated by sharing test failures

Test-driven iteration is an iterative refinement technique (task statement 3.5), not a substitute for upfront planning. The root cause is skipping plan mode for a complex task, not the absence of tests. The existing test runner itself was the source of the incompatibility.

B The developer should have provided more detailed upfront instructions to direct execution specifying the exact migration pattern for every file type

The core issue is not insufficient instructions — it is that the developer did not explore the codebase to understand dependencies before choosing a strategy. More detailed instructions would still have used the wrong strategy because the test runner conflict was not known upfront.

C The developer should have used the interview pattern to have Claude ask questions about the migration before starting

The interview pattern helps surface considerations the developer might miss, but the fundamental issue is choosing direct execution for a task that requires codebase exploration and strategy evaluation. Plan mode, not just questioning, was needed to map dependencies and test impacts.

A The developer should have used plan mode to explore the codebase and evaluate migration strategies before committing to an approach

An 80+ file migration with multiple valid strategies and cross-cutting concerns (build tooling, test configuration, package.json) is a textbook plan mode scenario. Plan mode would have revealed the test runner incompatibility before any files were changed. Starting with direct execution for a complex, multi-approach task risks costly rework.

WHERE THIS COMES FROM

Lesson 3.4: Plan Mode vs Direct Execution (Plan mode)

Lesson 3.4: Plan Mode vs Direct Execution (Recognising complexity)